

 S&P 500 Data Engineering Project

Final Data Science Project Submission

Step 0

Team Members

1. Arijit Chakma
2. MD Moksudul Haque
3. MD Alamgir Hossain

```

1 # --- STEP 0: ENVIRONMENT SETUP ---
2 import os
3 from google.colab import drive, userdata
4
5 # 1. Mount Google Drive for persistent CSV storage
6 drive.mount('/content/drive')
7
8 # 2. Define your project path
9 BASE_PATH = '/content/drive/My Drive/SP500_Project/'
10 if not os.path.exists(BASE_PATH):
11     os.makedirs(BASE_PATH)
12     print(f"Created project folder: {BASE_PATH}")
13
14 # 3. Securely retrieve the API Key from Colab Secrets
15 try:
16     API_KEY = userdata.get('AV_API_KEY')
17     print("Alpha Vantage API Key successfully loaded from Secrets.")
18 except Exception as e:
19     print("ERROR: Please add 'AV_API_KEY' to your Colab Secrets (Key icon).")
20     API_KEY = None
21     print(f"Details: {e}")
22
23 # 4. Install external libraries
24 !pip install alpha_vantage mplfinance bs4 lxml
25
26 from alpha_vantage.timeseries import TimeSeries
27
28 # 5. Import libraries
29 from matplotlib import style
30 import bs4 as bs
31 import matplotlib.pyplot as plt
32 import mplfinance as mpf
33 import numpy as np
34 import pandas as pd
35 import pickle
36 import requests
37 import time
38

```

```

Mounted at /content/drive
Alpha Vantage API Key successfully loaded from Secrets.
Collecting alpha_vantage
  Downloading alpha_vantage-3.0.0-py3-none-any.whl.metadata (12 kB)
Collecting mplfinance
  Downloading mplfinance-0.12.10b0-py3-none-any.whl.metadata (19 kB)
Collecting bs4
  Downloading bs4-0.0.2-py2.py3-none-any.whl.metadata (411 bytes)
Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (6.0.3)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from alpha_vantage) (3.13.5)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from alpha_vantage) (2.32.4)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from mplfinance) (3.10.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from mplfinance) (2.2.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from bs4) (4.13.5)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (2.6.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (1.3.2)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (6.4.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->alpha_vantage) (1.18.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->bs4) (2.8.3)

```

```
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->bs4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (1.
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (0.12.1
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (4
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (1
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (26
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance) (3
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mplfinance)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->mplfinance) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->mplfinance) (2026.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->alpha_var
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->alpha_vantage) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->alpha_vantage)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->alpha_vantage)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->n
Downloading alpha_vantage-3.0.0-py3-none-any.whl (35 kB)
Downloading mplfinance-0.12.10b0-py3-none-any.whl (75 kB)
75.0/75.0 kB 5.1 MB/s eta 0:00:00
Downloading bs4-0.0.2-py2.py3-none-any.whl (1.2 kB)
Installing collected packages: bs4, mplfinance, alpha_vantage
Successfully installed alpha_vantage-3.0.0 bs4-0.0.2 mplfinance-0.12.10b0
```

Questions for Step 0

1. Why is it considered a security risk to hardcode your API key directly into a script?

Hardcoding an API key is a security risk because anyone who can see the notebook or source code can copy and misuse the key. That can lead to unauthorized requests, rate-limit abuse, or even account suspension. A more professional practice is to store secrets securely in tools like Colab Secrets or environment variables.

2. What happens to files saved in the default `/content/` directory of a Google Colab notebook after the session ends?

Files saved in `/content/` are temporary. When the Colab runtime disconnects, restarts, or the session ends, those files are usually deleted. This means they should not be used for long-term project storage.

3. Explain the purpose of `drive.mount('/content/drive')` in a professional data pipeline.

`drive.mount('/content/drive')` connects Google Drive storage to the Colab notebook so files can be read from and saved to persistent cloud storage. In a professional data pipeline, this is useful because raw data, processed files, and outputs remain available across notebook sessions instead of being lost when the temporary runtime resets.

Step 1 : ROBUST DATA ACQUISITION

```
1 # 1. SETUP API OBJECT
2 # Use the API_KEY and BASE_PATH established in Step 0
3 ts = TimeSeries(key=API_KEY, output_format='pandas')
4
5 def fetch_and_audit(symbol='TSLA'):
6     """
7     Fetches stock data, performs a metadata audit, and validates data integrity.
8     """
9     try:
10         print(f"Initiating API request for {symbol}...")
11
12         # Request daily stock data using the compact output size
13         data, meta_data = ts.get_daily(symbol=symbol, outputsize='compact')
14
15         # METADATA AUDIT
16         print("\n---Metadata Audit ---")
17         for key, value in meta_data.items():
18             print(f"{key}: {value}")
19
20         # DATA INTEGRITY CHECK
21         print("\n-- Data Integrity Report ---")
22         print(f"Total Rows: {len(data)}")
23         print("Missing values by column:")
24         print(data.isnull().sum())
25
26         # Sort chronologically for downstream steps
27         data.sort_index(inplace=True)
28
29         # PREVIEW DATA
30         print("\n---Most Current Market Entries ---")
31         print(data.tail(5))
32
```

```

33     # DATA PERSISTENCE
34     save_path = os.path.join(BASE_PATH, f"{symbol}_daily_raw.csv")
35     data.to_csv(save_path)
36     print(f"\n Saved raw data to: {save_path}")
37
38     return data
39
40 except Exception as e:
41     # Basic error handling for rate limits
42     error_message = str(e).lower()
43     print("\n API ERROR:")
44     if "call frequency" in error_message or "premium" in error_message:
45         print("The Alpha Vantage API rate limit was reached. Please wait and try again later.")
46     else:
47         print(str(e))
48     return None
49
50 # Execute Step 1
51 df_raw = fetch_and_audit('TSLA')
52

```

Initiating API request for TSLA...

---Metadata Audit ---

```

1. Information: Daily Prices (open, high, low, close) and Volumes
2. Symbol: TSLA
3. Last Refreshed: 2026-04-17
4. Output Size: Compact
5. Time Zone: US/Eastern

```

-- Data Integrity Report ---

```

Total Rows: 100
Missing values by column:
1. open      0
2. high      0
3. low       0
4. close     0
5. volume    0
dtype: int64

```

---Most Current Market Entries ---

	1. open	2. high	3. low	4. close	5. volume
date					
2026-04-13	350.065	356.35	348.57	352.42	53617477.0
2026-04-14	357.670	367.63	354.77	364.20	59979607.0
2026-04-15	366.830	394.65	362.50	391.95	113810355.0
2026-04-16	393.810	394.06	381.80	388.90	63515136.0
2026-04-17	395.920	409.28	391.65	400.62	90640032.0

Saved raw data to: /content/drive/My Drive/SP500_Project/TSLA_daily_raw.csv

Questions for Step 1

1. What is the difference between the `get_daily` and `get_intraday` endpoints in the Alpha Vantage API?

`get_daily` returns one record per trading day, which is useful for daily trend analysis over weeks, months, or years.

`get_intraday` returns market data at smaller intervals such as 1-minute, 5-minute, or 60-minute periods, which is better for short-term or within-day analysis.

2. Why is it important to print and review the `meta_data` dictionary before processing the actual stock prices?

The `meta_data` dictionary confirms important context such as the symbol, last refreshed time, output size, and time zone.

Reviewing it helps verify that the API returned the correct dataset before analysis begins, which reduces the risk of using stale, incomplete, or incorrect data.

3. If the API returns a "Rate Limit" error, what code structure can you use to handle it without the script crashing?

A `try-except` block should be used so the script catches the exception instead of crashing. Inside the `except` block, the error message can be checked for phrases such as `"call frequency"` or `"premium"`, and the script can print a user-friendly message, wait, or retry later.

Step 2: PLOTTING & INDEXING

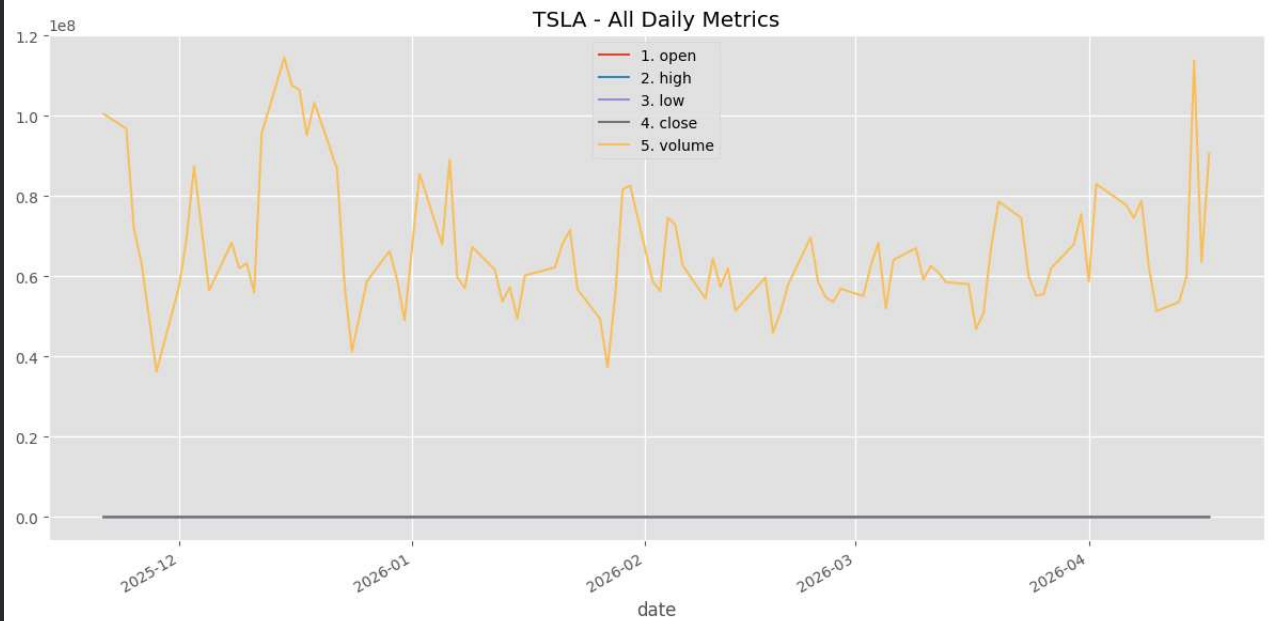
```

1 # Set visual style
2 style.use('ggplot')
3
4 def process_and_plot(symbol='TSLA'):
5     """

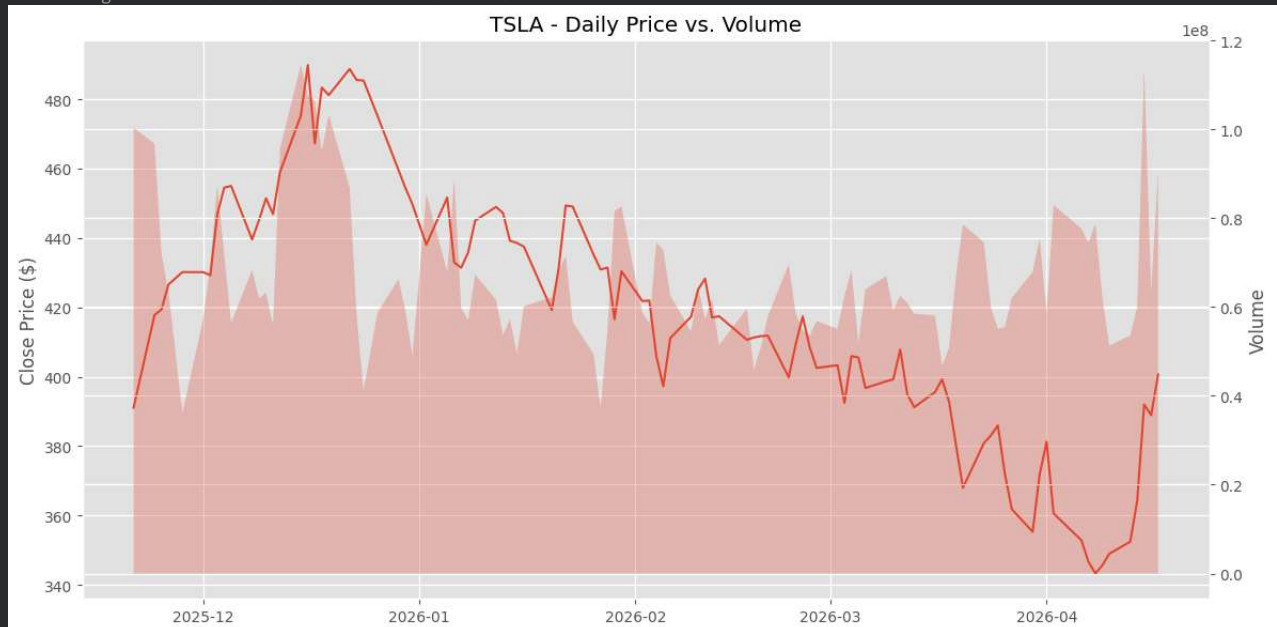
```

```
6     Cleans the raw CSV data and generates multi-axis visualizations.
7     """
8     # 1. LOAD DATA
9     raw_path = os.path.join(BASE_PATH, f'{symbol}_daily_raw.csv')
10
11     if not os.path.exists(raw_path):
12         print("Error: CSV not found. Please run Step 1 first.")
13         return
14
15     # Load data with a DatetimeIndex
16     df = pd.read_csv(raw_path, index_col='date', parse_dates=True)
17
18     # DATA CLEANING
19     df.sort_index(inplace=True)
20
21     # COMPREHENSIVE PLOT
22     print(f"Generating basic comprehensive chart for {symbol}...")
23     df.plot(figsize=(12, 6), title=f"{symbol} - All Daily Metrics")
24     plt.tight_layout()
25     plt.show()
26
27     # DUAL-AXIS VISUALIZATION
28     print(f"Generating Dual-Axis Price vs Volume chart for {symbol}...")
29     fig, ax1 = plt.subplots(figsize=(12, 6))
30
31     ax1.plot(df.index, df['4. close'], label='Close Price')
32     ax1.set_ylabel('Close Price ($)')
33
34     ax2 = ax1.twinx()
35     ax2.fill_between(df.index, df['5. volume'], alpha=0.3)
36     ax2.set_ylabel('Volume')
37
38     plt.title(f"{symbol} - Daily Price vs. Volume")
39     plt.tight_layout()
40     plt.show()
41
42 # Execute Step 2
43 process_and_plot('TSLA')
44
```

Generating basic comprehensive chart for TSLA...



Generating Dual-Axis Price vs Volume chart for TSLA...



Questions for Step 2

1. Why do we need to sort the index of the DataFrame using `df.sort_index(inplace=True)` before plotting?

The API often returns stock records in reverse chronological order, with the newest dates first. Sorting the index places the data in true time order from oldest to newest so the line chart is drawn correctly and time-based analysis becomes easier to understand.

2. What visualization problem occurs when you plot "Close Price" and "Volume" on the same Y-axis, and how do you fix it?

Volume values are usually much larger than stock prices, so plotting both on one Y-axis compresses the price line and makes it look almost flat. The professional fix is to use a secondary Y-axis with `twinx()` so price and volume each have their own scale.

3. What is the benefit of setting the DataFrame index to a `DatetimeIndex` instead of leaving it as a standard string?

A `DatetimeIndex` allows pandas to treat the data as a time series. This makes it easier to sort dates, filter by time period, resample data, roll moving averages, and display a proper timeline on plots.

Step 3: MOVING AVERAGES

```

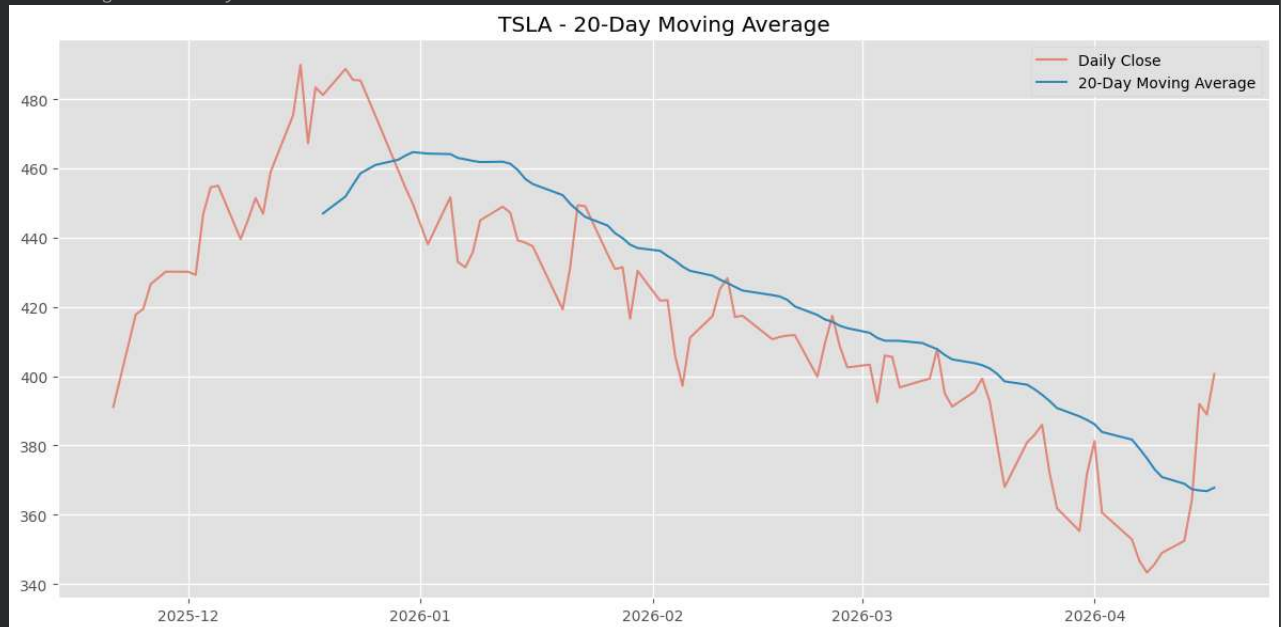
1 # --- STEP 3: TECHNICAL INDICATORS ---
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 import os
5
6 def calculate_trend(symbol='TSLA'):
7     """
8     Calculates technical indicators and overlays them on price data.
9     """
10    # 1. LOAD DATA
11    file_path = os.path.join(BASE_PATH, f'{symbol}_daily_raw.csv')
12
13    if not os.path.exists(file_path):
14        print("Error: Previous data not found.")
15        return
16
17    df = pd.read_csv(file_path, index_col='date', parse_dates=True)
18    df.sort_index(inplace=True)
19
20    # 2. CALCULATE MOVING AVERAGE
21    df['ma_line'] = df['4. close'].rolling(window=20).mean()
22
23    # DATA INSPECTION
24    print("\n--- Moving Average Preview (First 25 Rows) ---")
25    print(df[['4. close', 'ma_line']].head(25))
26
27    # 3. VISUALIZATION
28    print(f"Generating Trend Analysis chart for {symbol}...")
29    plt.figure(figsize=(12, 6))
30    plt.plot(df.index, df['4. close'], label='Daily Close', alpha=0.6)
31    plt.plot(df.index, df['ma_line'], label='20-Day Moving Average')
32    plt.title(f"{symbol} - 20-Day Moving Average")
33    plt.legend()
34    plt.tight_layout()
35    plt.show()
36
37 # Execute Step 3
38 calculate_trend('TSLA')
39

```

```
--- Moving Average Preview (First 25 Rows) ---
```

date	4. close	ma_line
2025-11-21	391.09	NaN
2025-11-24	417.78	NaN
2025-11-25	419.40	NaN
2025-11-26	426.58	NaN
2025-11-28	430.17	NaN
2025-12-01	430.14	NaN
2025-12-02	429.24	NaN
2025-12-03	446.74	NaN
2025-12-04	454.53	NaN
2025-12-05	455.00	NaN
2025-12-08	439.58	NaN
2025-12-09	445.17	NaN
2025-12-10	451.45	NaN
2025-12-11	446.89	NaN
2025-12-12	458.96	NaN
2025-12-15	475.31	NaN
2025-12-16	489.88	NaN
2025-12-17	467.26	NaN
2025-12-18	483.37	NaN
2025-12-19	481.20	446.9870
2025-12-22	488.73	451.8690
2025-12-23	485.56	455.2580
2025-12-24	485.40	458.5580
2025-12-26	475.19	460.9885
2025-12-29	459.64	462.4620

Generating Trend Analysis chart for TSLA...



Questions for Step 3

1. Why do the first several rows of a new "Moving Average" column contain NaN values?

The first rows contain `NaN` because a rolling average needs a full window of earlier observations before it can calculate a value. For a 20-day moving average, pandas needs 20 closing prices, so the first 19 rows do not yet have enough data.

2. How does a Moving Average help a financial analyst compared to looking at the raw daily closing price?

A moving average smooths out short-term fluctuations and random daily noise, making the broader market trend easier to see. This helps analysts identify direction, momentum, and possible support or resistance zones more clearly than raw daily prices alone.

3. What does the window parameter in `df.rolling(window=X).mean()` represent?

The `window` parameter represents the number of observations included in each rolling calculation. For example, `window=20` means each moving average point is based on the most recent 20 rows of data.

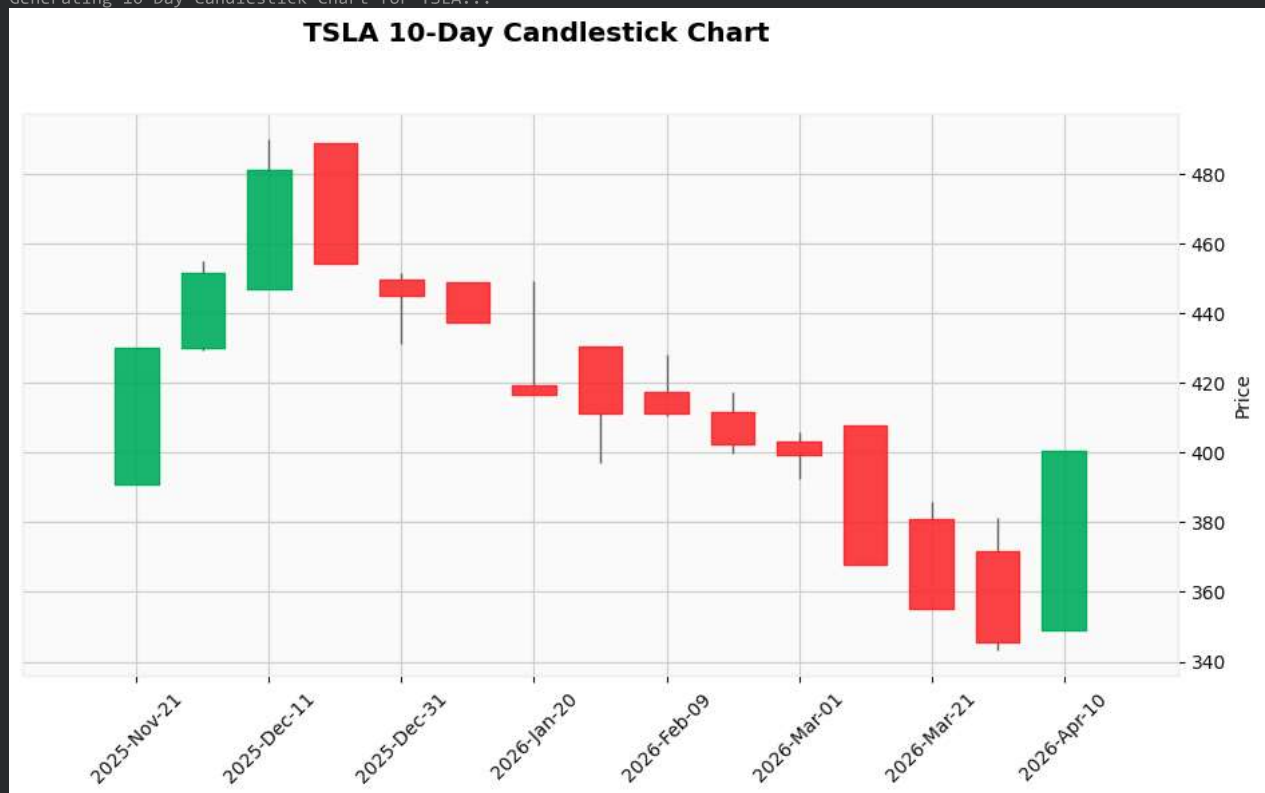
Step 4a: CANDLESTICK CHARTS & RESAMPLING

```

1 def generate_candlesticks(symbol='TSLA'):
2     """
3     Transforms daily data into 10-day OHLC bars and plots candlesticks.
4     """
5     # 1. LOAD DATA
6     raw_path = os.path.join(BASE_PATH, f'{symbol}_daily_raw.csv')
7     if not os.path.exists(raw_path):
8         print("Error: Raw data CSV not found.")
9         return
10
11     df = pd.read_csv(raw_path, index_col='date', parse_dates=True)
12     df.sort_index(inplace=True)
13
14     # 2. DATA RESAMPLING
15     ohlc_df = df['4. close'].resample('10D').ohlc()
16
17     # Rename columns for mplfinance compatibility
18     ohlc_df.columns = ['Open', 'High', 'Low', 'Close']
19
20     # 3. VISUALIZATION
21     print(f"Generating 10-Day Candlestick Chart for {symbol}...")
22     mpf.plot(
23         ohlc_df,
24         type='candle',
25         style='yahoo',
26         title=f'{symbol} 10-Day Candlestick Chart',
27         figsize=(12, 6)
28     )
29
30 # Execute Step 4a
31 generate_candlesticks('TSLA')
32

```

Generating 10-Day Candlestick Chart for TSLA...



Step 4b: WEB SCRAPING & PICKLING

```

1 def save_sp500_tickers():
2     """
3     Scrapes Wikipedia for S&P 500 tickers and saves them as a pickle file.

```

```

4     """
5     print("Scraping S&P 500 ticker list from Wikipedia...")
6
7     # 1. FETCH THE WEBPAGE
8     resp = requests.get("https://en.wikipedia.org/wiki/List_of_S%26P_500_companies", headers={"User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0"})
9
10    # 2. PARSE HTML
11    soup = bs.BeautifulSoup(resp.text, "lxml")
12
13    # Find the table with the id 'constituents'
14    table = soup.find("table", {"id": "constituents"})
15
16    tickers = []
17
18    # 3. EXTRACT DATA
19    for row in table.find_all("tr")[1:]:
20        cols = row.find_all("td")
21        if cols:
22            ticker = cols[0].text.strip().replace(".", "-")
23            tickers.append(ticker)
24
25    # 4. DATA PERSISTENCE
26    pickle_path = os.path.join(BASE_PATH, "sp500tickers.pickle")
27    with open(pickle_path, "wb") as f:
28        pickle.dump(tickers, f)
29
30    print(f"Successfully saved {len(tickers)} tickers.")
31    return tickers
32
33 # Execute Step 4b
34 tickers_list = save_sp500_tickers()
35

```

Scraping S&P 500 ticker list from Wikipedia...
Successfully saved 503 tickers.

Questions for Step 4

1. What information does a single "Candlestick" provide that a standard line chart does not?

A single candlestick shows four pieces of price information for a time period: open, high, low, and close. A standard line chart usually shows only one value, such as the closing price, so it does not reveal the full trading range or whether price moved up or down within that period.

2. Explain the process of "Resampling" data from 1-day to 10-day intervals.

Resampling groups smaller time intervals into larger ones. In this project, daily closing-price data is grouped into 10-day periods and converted into OHLC values, where the first value becomes the open, the maximum becomes the high, the minimum becomes the low, and the last value becomes the close for each 10-day block.

3. Why is it beneficial to "Pickle" the S&P 500 ticker list after scraping it from Wikipedia?

Pickling the ticker list saves the scraped results locally so the program can reload them quickly later without scraping the website again. This improves efficiency, reduces unnecessary web requests, and keeps the pipeline more reproducible.

Step 5: AUTOMATED PIPELINE

```

1 # STEP 5: AUTOMATED PIPELINE ---
2 from alpha_vantage.timeseries import TimeSeries
3 import pandas as pd
4 import os
5 import time
6 import pickle
7
8 # 1. SETUP API & PATHS
9 ts = TimeSeries(key=API_KEY, output_format='pandas')
10 DATA_DIR = os.path.join(BASE_PATH, 'stock_dfs/')
11
12 # Ensure the DATA_DIR directory exists before proceeding
13 if not os.path.exists(DATA_DIR):
14     os.makedirs(DATA_DIR)
15     print(f"Created data directory: {DATA_DIR}")
16
17 def compile_data(reload_sp500=False):
18     """
19     Automates data collection for 25 tickers and compiles them into a master CSV.

```

```
20 """
21 # 2. LOAD TICKER LIST
22 ticker_path = os.path.join(BASE_PATH, 'sp500tickers.pickle')
23 if reload_sp500 or not os.path.exists(ticker_path):
24     save_sp500_tickers()
25
26 with open(ticker_path, 'rb') as f:
27     tickers = pickle.load(f)[:25]
28
29 # --- PART 1: DATA ACQUISITION LOOP ---
30 calls_made = 0
31 for ticker in tickers:
32     file_path = os.path.join(DATA_DIR, f"{ticker}.csv")
33
34     # Check if the CSV already exists in DATA_DIR
35     if not os.path.exists(file_path):
36         try:
37             print(f"Downloading data for {ticker}...")
38             df, _ = ts.get_daily(symbol=ticker, outputsize='compact')
39             df.sort_index(inplace=True)
40             df.to_csv(file_path)
41             calls_made += 1
42             time.sleep(15)
43         except Exception as e:
44             print(f"Failed to fetch {ticker}: {e}")
45     else:
46         print(f"Using existing file for {ticker}")
47
48 # PART 2: DATA COMPILATION ---
49 print("\nCompiling master dataframe...")
50 main_df = pd.DataFrame()
51
52 for ticker in tickers:
53     file_path = os.path.join(DATA_DIR, f"{ticker}.csv")
54
55     if os.path.exists(file_path):
56         temp_df = pd.read_csv(file_path, index_col='date', parse_dates=True)
57         temp_df = temp_df[['4. close']].rename(columns={'4. close': ticker})
58
59         if main_df.empty:
60             main_df = temp_df
61         else:
62             main_df = main_df.join(temp_df, how='outer')
63
64 # 3. SAVE THE MASTER FILE
65 master_path = os.path.join(BASE_PATH, 'sp500_joined_closes.csv')
```